

How hard can it be to count a few XML nodes?

Devolutions



Jane Street



Microsoft



SynEdgy



@schusterfredl.bsky.social

Performance Myth-Busting

- “language X” is slow to do <something>
- But actually
 - the library / implementation is key

Why I Care

- Dev in Automotive Engineering
 - configuration and measurement-Data parsers
- C++ / XSLT / C# / Rust / PowerShell
- “xml-i” – personal pet-project to test assumptions and experiment

Same CLI, Many Implementations

- xml-i (“xml eye”): count XML node names

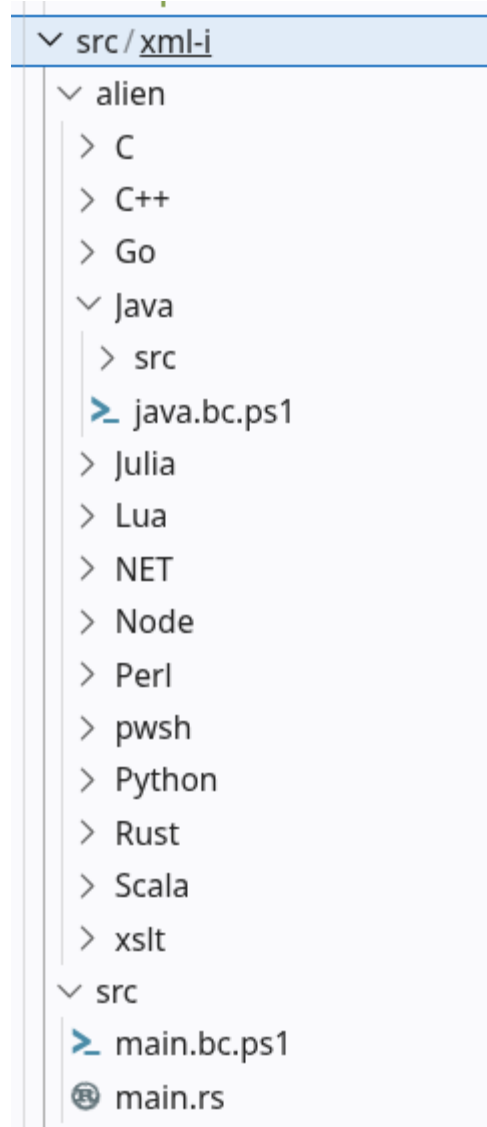
```
./target/release/xml-i ./test/huge.xml boing blips
```

Node counts:

boing: 1342440

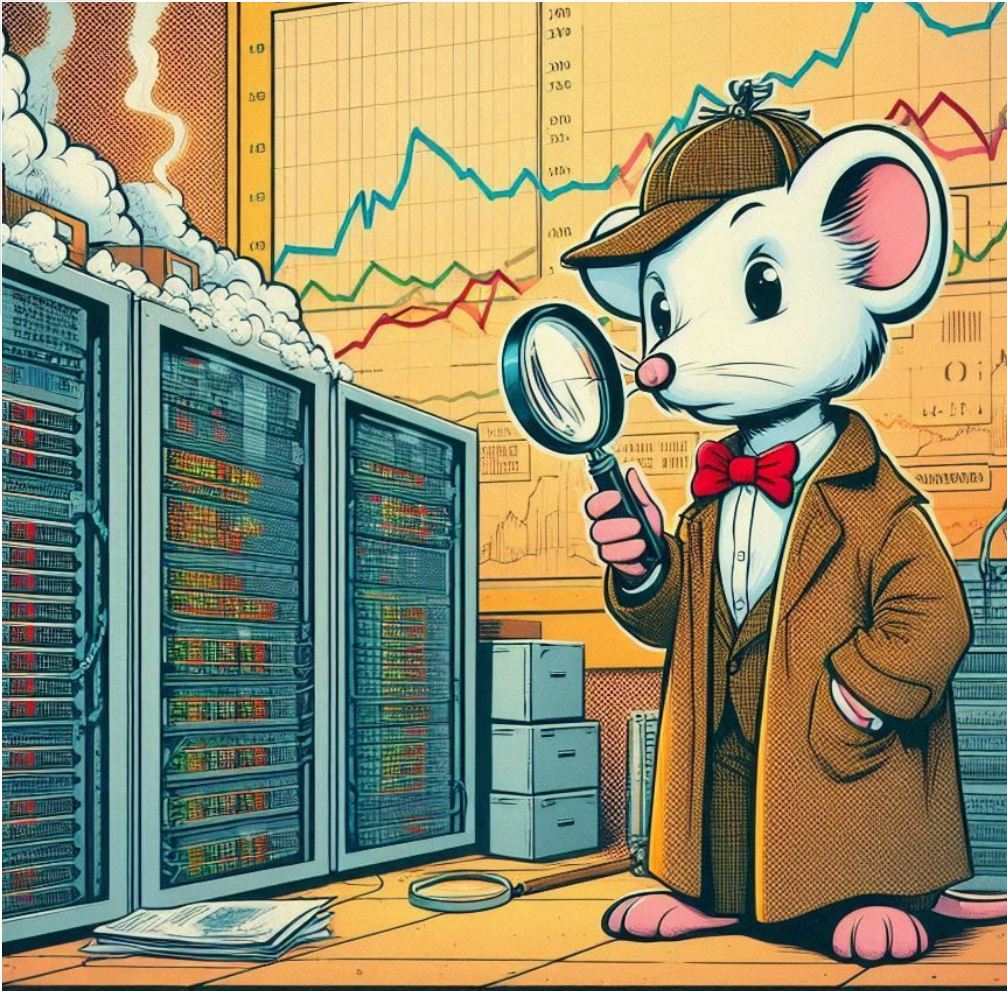
Blips: 1279

The “alien” Folder



- `./src/*` → the baseline
- `./alien/*` → different languages/implementations

Benchmarking



- reproducible setup
 - same test file
 - same language/stacks
- I/O cold-start
 - `/proc/sys/vm/drop_caches`
- comparable

Will It Scale?

500kB file → boring

1 MB file → easy

100MB file → it gets interesting

1 GB file → RAM becomes a liability

3GB file → "**Behemoth**"

(possible, but rarely seen)

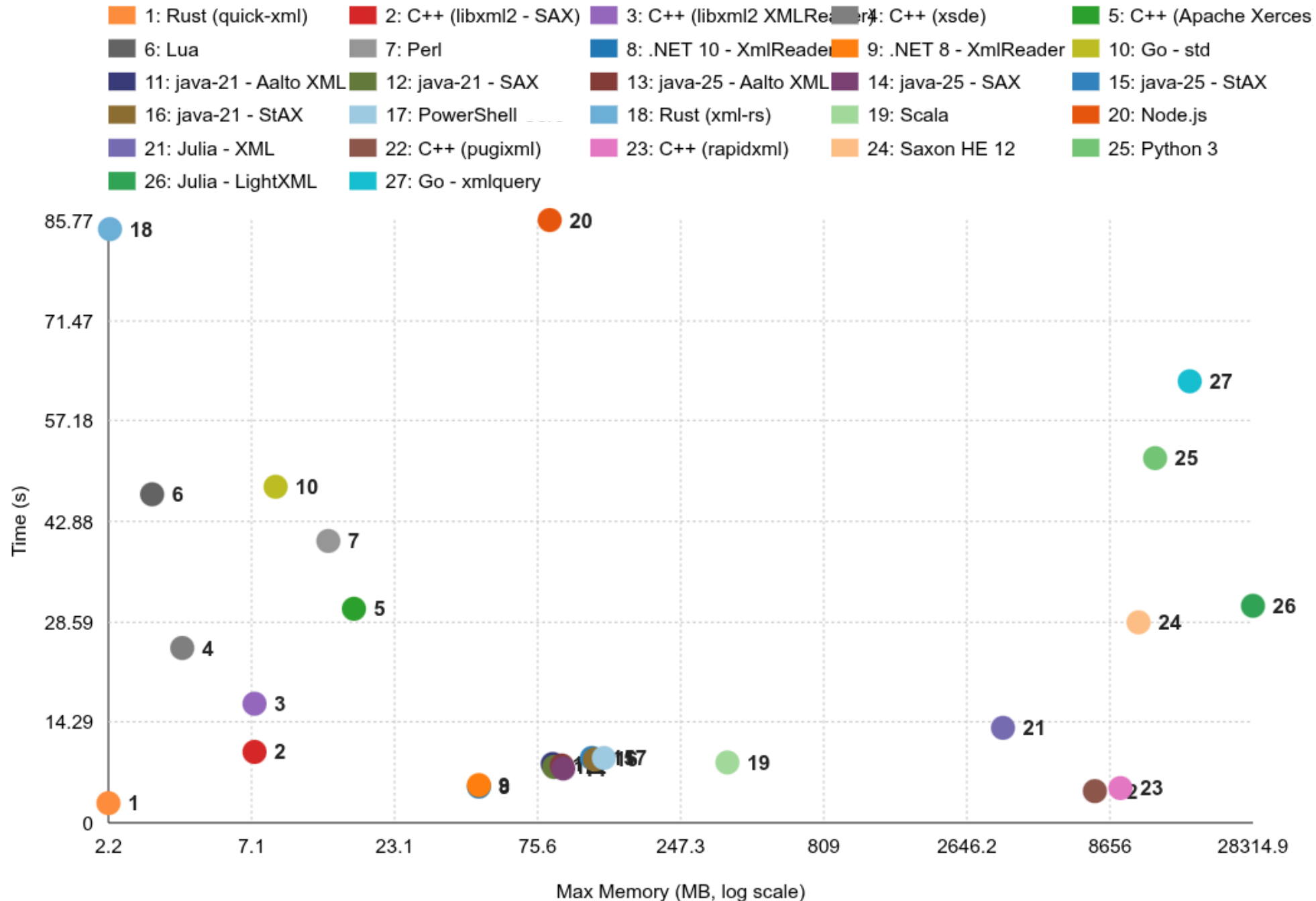
Different Parsing Mindsets

- DOM
 - Load whole structure into memory
- SAX
 - Stream through content, callbacks to caller
- StAX
 - Stream through content by pulling tokens
- ...

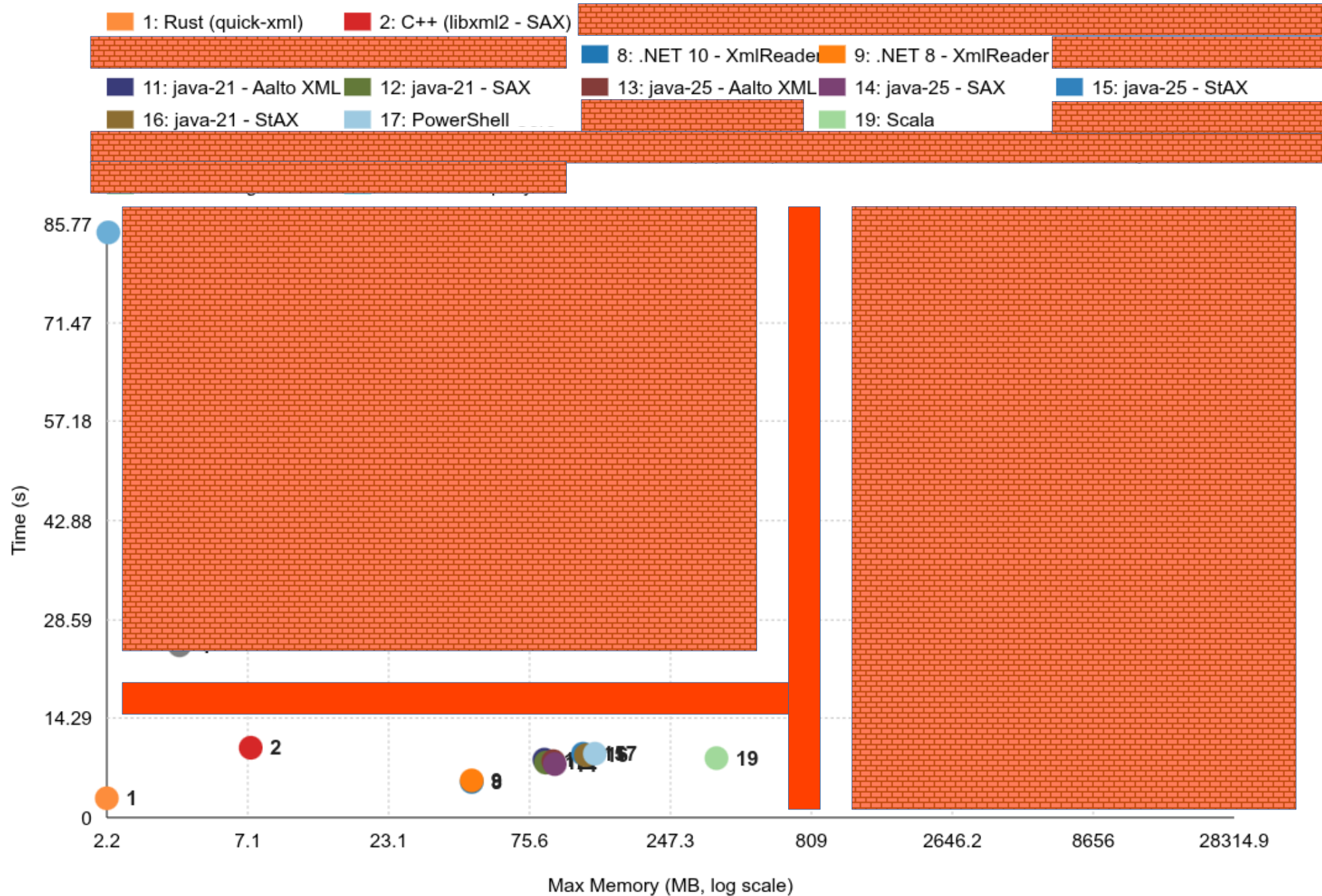
...

- DOM
 - Load *whole structure into memory*
- SAX
 - Stream through content, *callbacks to caller*
- **StAX**
 - Stream through content by **pulling tokens**
- ...

Time vs. Max Memory (behemoth.xml - 3.16GB)

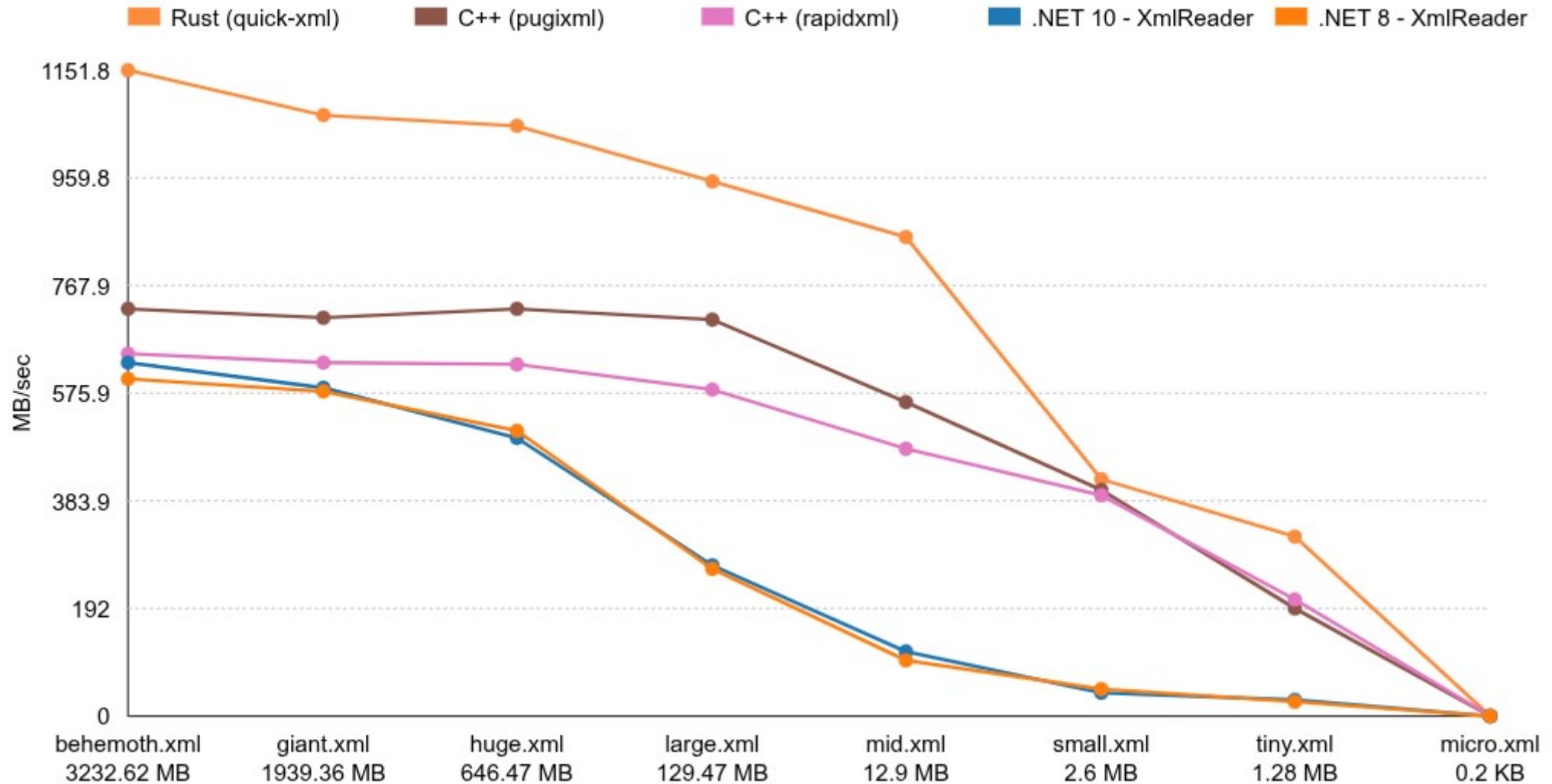


Time vs. Max Memory (behemoth.xml - 3.16GB)



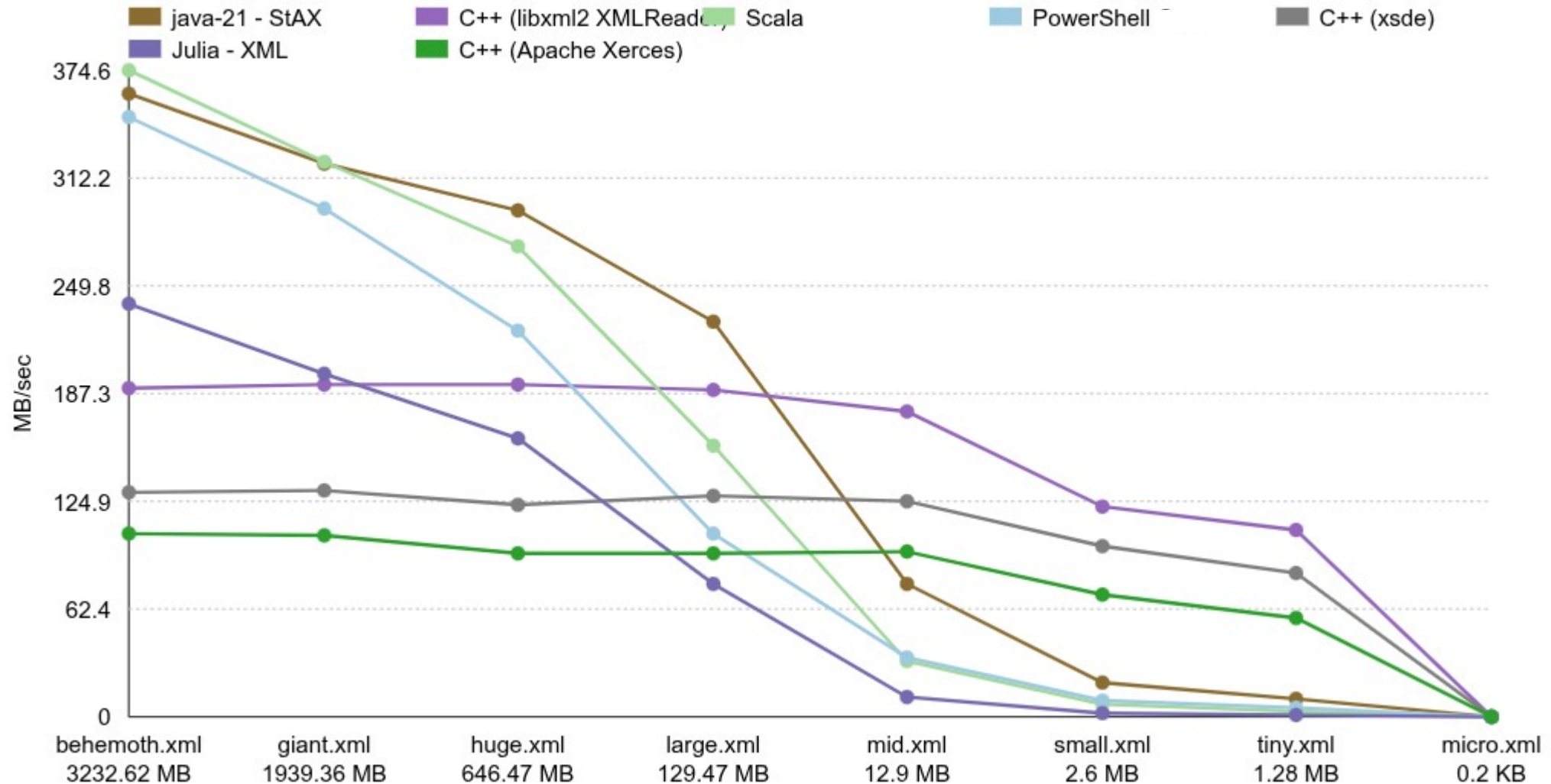
Throughput Leaders

Max. Throughput (top 5)



PowerShell Surprise

Max. Throughput (around PowerShell Core (+/-3))



```
PSVersion           {[Major, 7], [Minor, 4], [Patch, 13],  
[PreReleaseLabel, ]}  
PSEdition           Core  
BuildVersion  
CLRVersion  
OS                  Manjaro Linux  
^^^
```

File	Time (s)	Throughput (MB/s)	Max Mem (MB)
behemoth.xml	9.292	347.874	130.10
giant.xml	6.595	294.065	135.34
huge.xml	2.885	224.096	135.29
large.xml	1.224	105.766	135.36
mid.xml	0.375	34.385	133.83
small.xml	0.278	9.344	131.05
tiny.xml	0.257	4.980	126.91
micro.xml	0.242	0.001	123.74

...

```
# do NOT: $content = [xml] (Get-Content -Raw -Path $XmlFilePath)

# stream it using .NET!
$reader = [System.Xml.XmlReader]::Create($XmlFilePath, $settings)

try {
    while ($reader.Read()) {
        if ($reader.NodeType -eq [System.Xml.XmlNodeType]::Element) {
            $nodeName = $reader.LocalName
            $nodeCounts[$nodeName]++
        }
    }
}
```


Convenience Wins

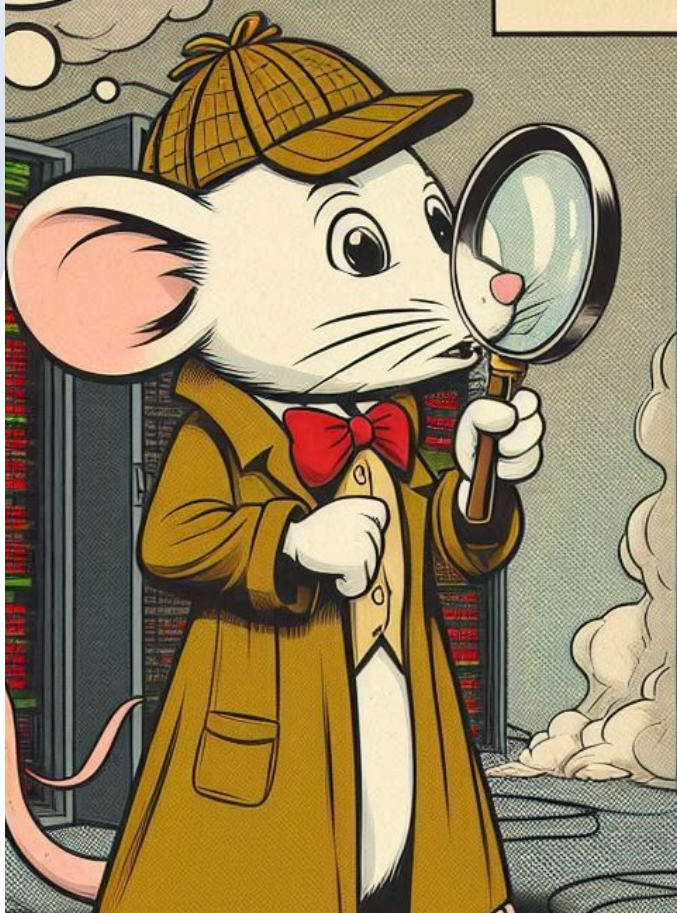
- [xml] (Get-Content -Raw ...)
 - one-shot
 - small files
 - startup > execution time



PowerShell

- Myth: PowerShell performance bad :-(
- Reality: Actually pretty competitive :-)

If it's slow...



... it's most likely on you :-)

Pick Your Bottleneck

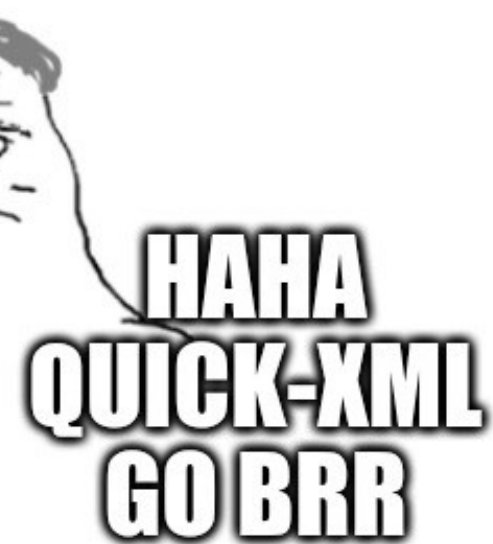
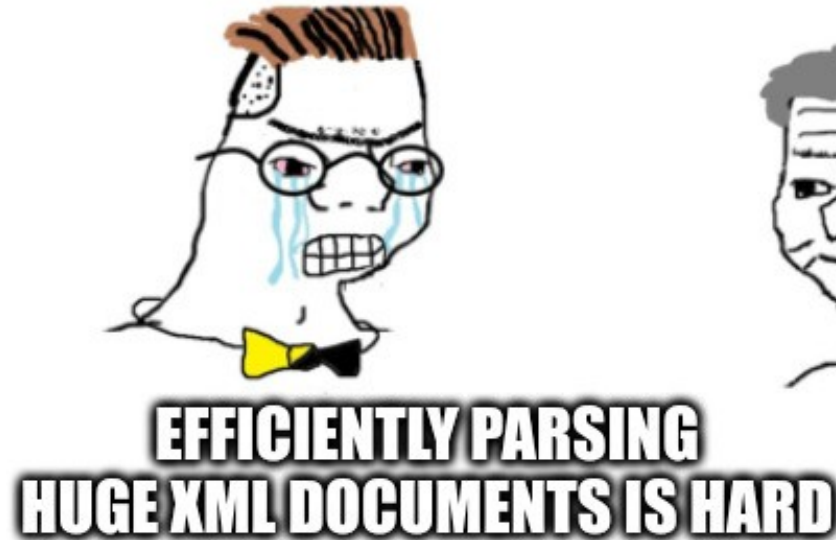
- Speed
- Memory
- Implementation overhead?

Call For Action

- Benchmark one real XML processing task
 - Time + Size => Throughput
 - Memory consumption
 - ***Does it (need to) scale?***

Thank You!

That's it from my side,
all data + code



- github.com/mwallner/xml-i
- github.com/mwallner/PSQuickXml